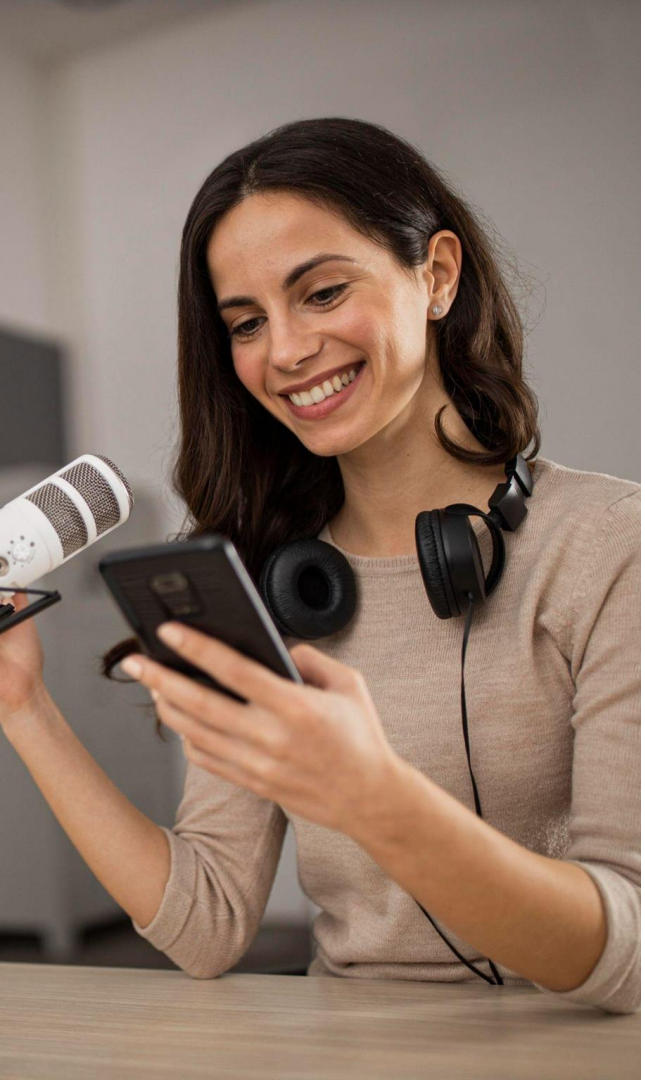


SISTEMAS OPERATIVOS SECCIÓN 10

ING. JUAN LUIS GARCIA Z.
UNIVERSIDAD DEL VALLE DE GUATEMALA

01 Procesos

ING. JUAN LUIS GARCIA

Un proceso como conjunto

I

INSTRUCCIONES

Las instrucciones del programa a punto de ser ejecutado.

E

ESTADO

Existen diferentes estados para verificar la ejecucion del estado.

D

DIRECCION DE MEMORIA

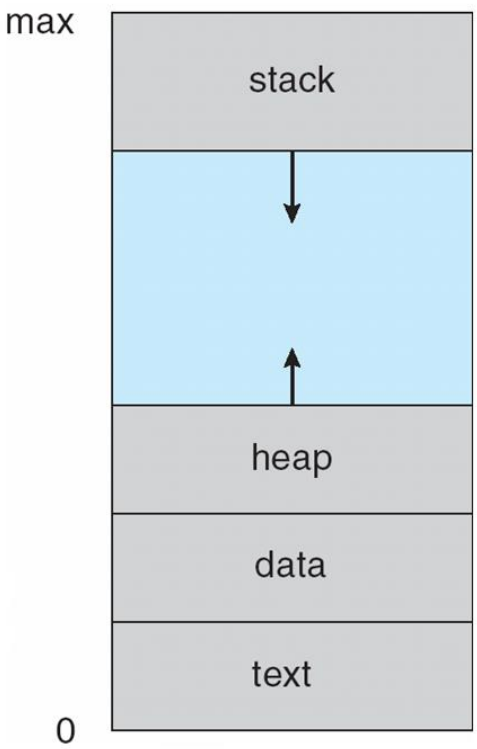
La direccion de memoria donde se esta ejecutando.

Disclaimer: Esto no aplica a sistemas multihilo en donde encontramos procesos que pueden compartir recursos entre ellos.

El procesos como unidad fundamental de un OS

Volvamos al ejemplo de un OS como nuestro Gobierno, como podriamos homologar a los procesos?

Como nos podemos imaginar un proceso?



Conceptualmente, cada proceso tiene su propio CPU por acarrearse en un contexto aislado.

Dos procesos separados pueden asociarse con un mismo programa.

Pila: almacenamiento temporal para operaciones administrativas (parámetros, variables locales, direcciones de retorno...)
Data: variables globales.
Text: código siendo ejecutado.
Heap: espacio de memoria asignada de forma dinámica, modificable por el usuario.

Estados de los procesos

El estado de un proceso cambia a lo largo de su ciclo de vida. Permite tomar decisiones sobre el manejo y la calendarización del proceso.

Nuevo: proceso creado o en creación, todavía no cargado a memoria.

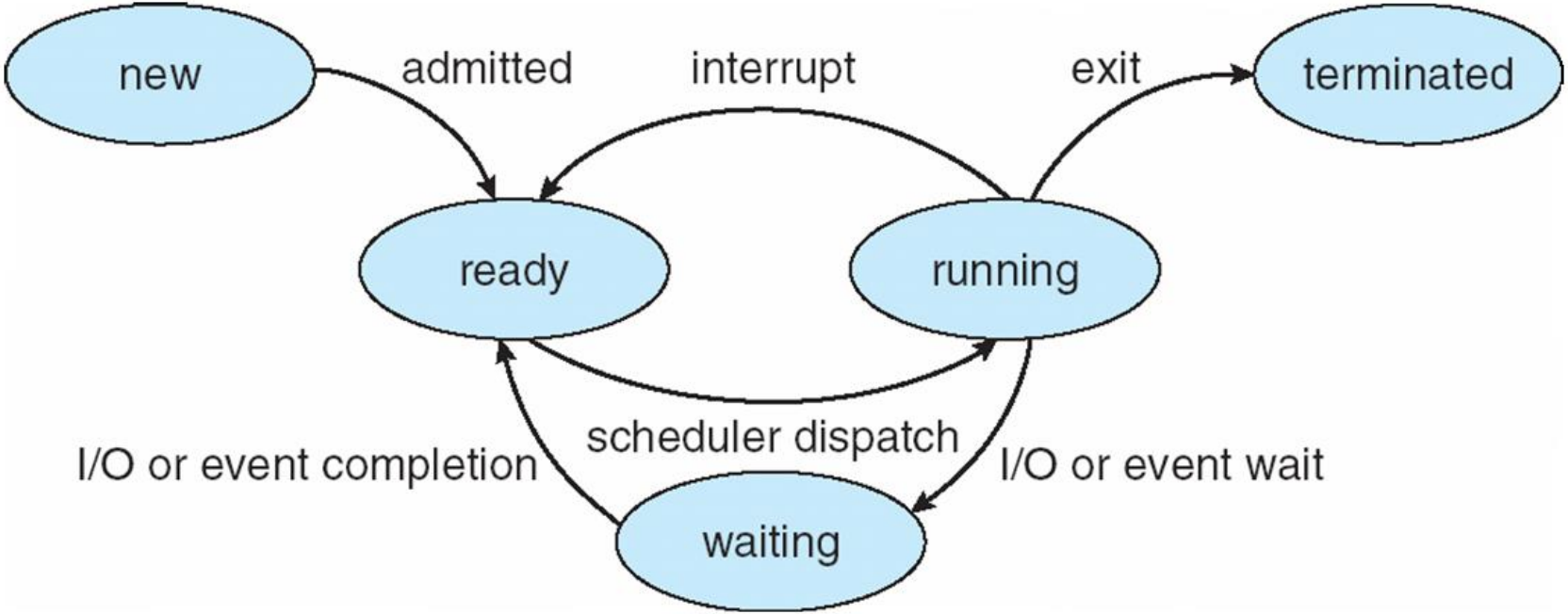
En ejecución: el proceso tiene procesador asignado y está corriendo.

Esperando/bloqueado: incapaz de continuar ejecutándose hasta que ocurra algún evento externo.

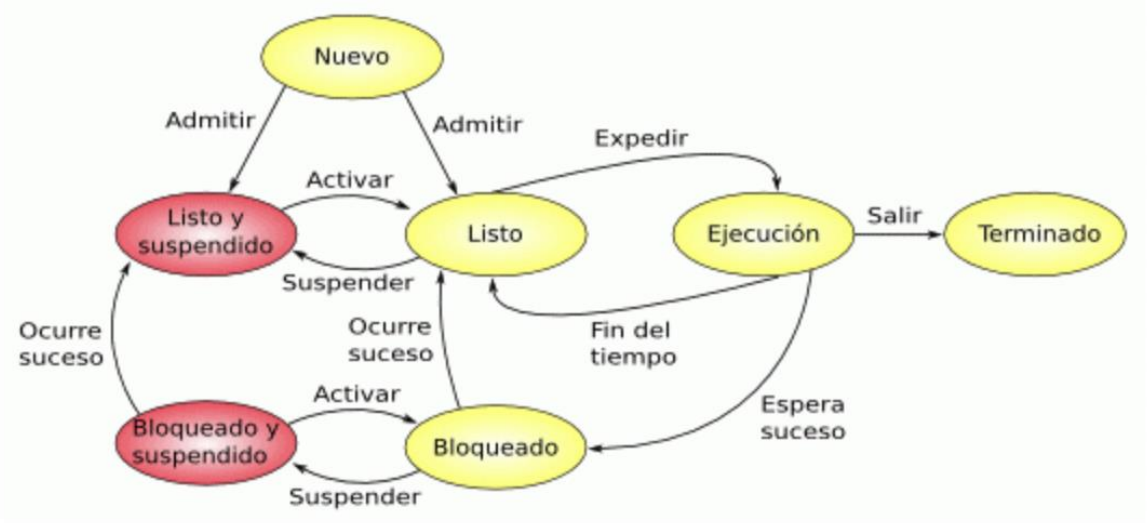
Listo: el proceso está en cola, esperando ejecución.

Terminado: proceso concluido, listo para ser eliminado.

Estados de los procesos

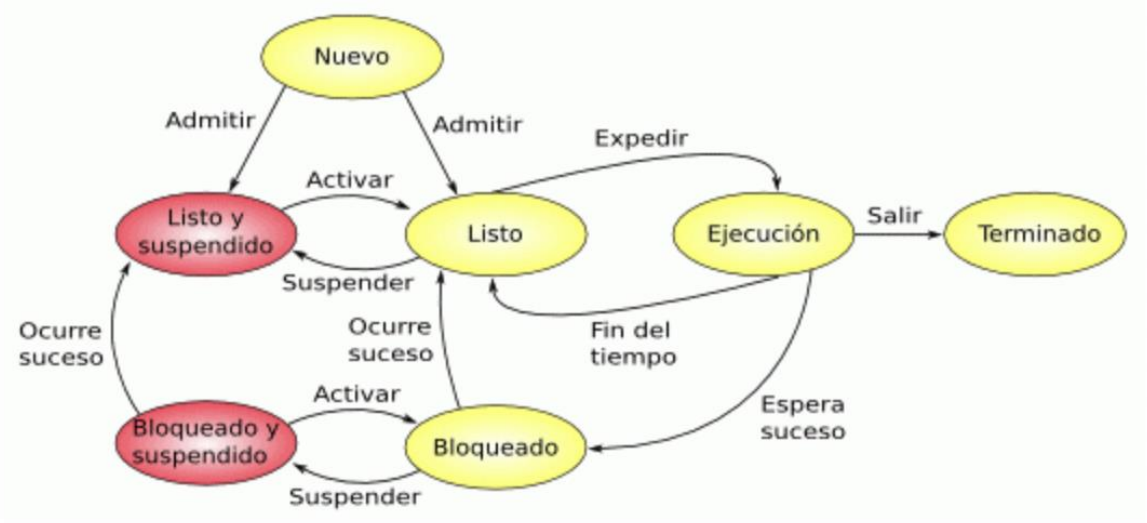


Algo complicado?



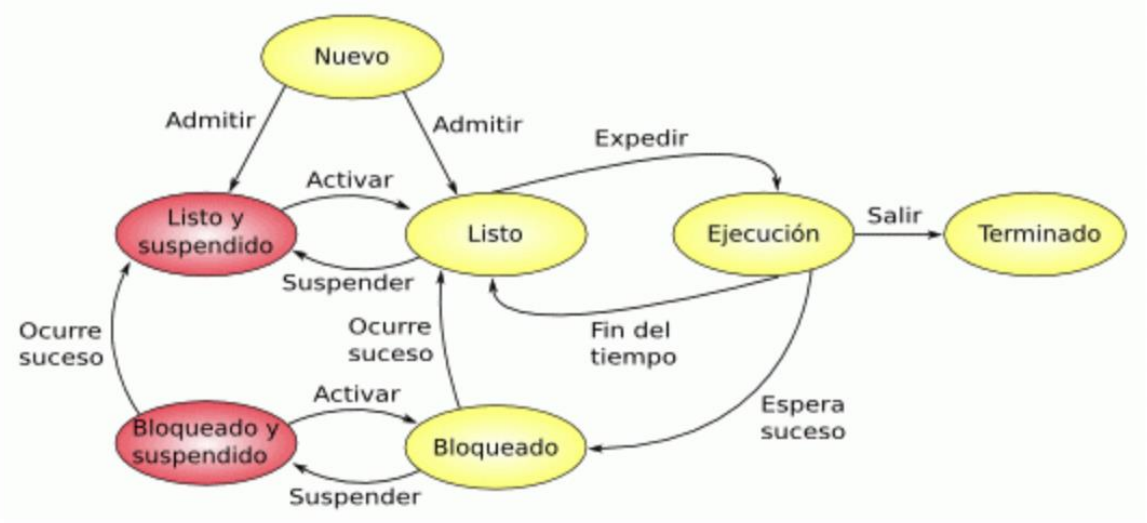
- Suspendido: Cuando un proceso está en estado suspendido, significa que ha sido temporalmente detenido por el sistema operativo y se encuentra en un estado inactivo.
- Bloqueado : Un proceso bloqueado se encuentra en un estado en el que está esperando la finalización de una operación de entrada/salida (E/S) para continuar su ejecución.

Algo complicado?



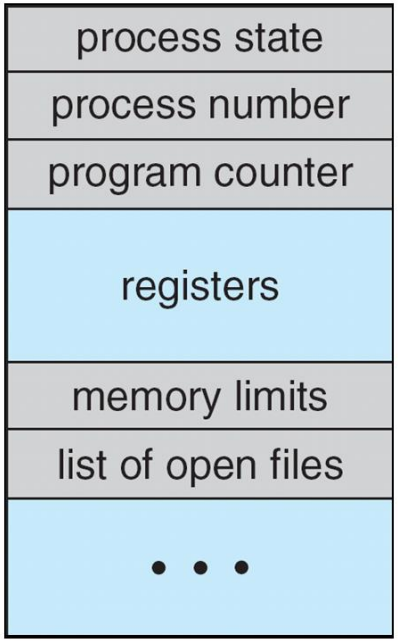
- En donde nos podria funcionar el estado bloqueado?
- En donde nos podria funcionar el estado suspendido?

Algo complicado?



- En donde nos podria funcionar el estado suspendido?
- **Al esperar un IO**
- En donde nos podria funcionar el estado suspendido?
- **Al darle mas privilegio a un proceso que necesite nuestros recursos**

Process Control Block **PCB**



El PCB es una estructura de datos fundamental en los sistemas operativos que se utiliza para almacenar y gestionar información específica de un proceso.

Las principales funciones del PCB incluyen:

1. Identificación del Proceso: Identificador único para el proceso, como un número de proceso o un identificador de control asignado por el sistema operativo.

2. Estado del Proceso:Indica el estado actual del proceso

3. Contador de Programa y Registros de la CPU: Guarda la posición actual del programa en ejecución, incluido el contador de programa y los valores de los registros de la CPU. Esto permite que el proceso se reanude exactamente desde donde se detuvo.

4. Limites de memoria: Localización de los espacios de memoria RAM que esta utilizando el proceso.

5. Información de E/S (Entrada/Salida):Contiene detalles sobre operaciones de entrada/salida pendientes y dispositivos utilizados por el proceso.

CAMBIO DE CONTEXTO

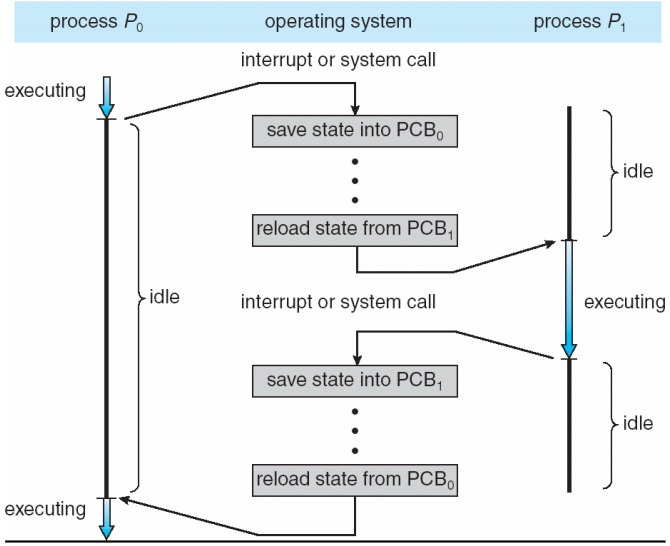
Se refiere al proceso de guardar el estado actual de un proceso que está siendo ejecutado en la CPU para luego cargar el estado de otro proceso y permitir que continúe su ejecución.
Cuando un sistema operativo ejecuta múltiples procesos simultáneamente, la CPU debe alternar entre ellos para dar la ilusión de ejecución **concurrente**.

1. Guardar el Estado del Proceso Actual: Se almacenan los registros de la CPU, el contador de programa y otra información relevante en el Bloque de Control de Proceso (PCB) del proceso actual. Esto se hace para que el sistema operativo pueda restaurar el estado exacto del proceso en el futuro.

2. Seleccionar el Próximo Proceso a Ejecutar: El sistema operativo elige cuál será el próximo proceso en ejecutarse. Esto puede basarse en políticas de planificación de procesos, prioridades, o simplemente en un esquema de tiempo compartido.

3. Cargar el Estado del Nuevo Proceso: Se recuperan los registros de la CPU y la información almacenada previamente en el PCB del próximo proceso a ejecutar. La CPU ahora comienza a ejecutar las instrucciones de este nuevo proceso.

CPU cambiando entre procesos



02 La responsabilidad de ser padre

ING. JUAN LUIS GARCIA

El nacimiento de un proceso

Inicio del OS

El OS crea procesos durante su inicialización, al proveer ciertos servicios y ante solicitudes del usuario.

Relacion Padre - Hijo

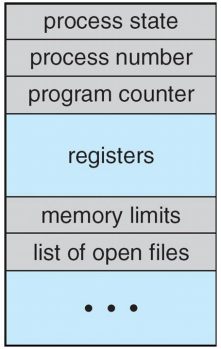
En sistemas Linux, los procesos de usuario pertenecen a un mismo árbol cuya raíz es el proceso init (o systemd).

fork ()

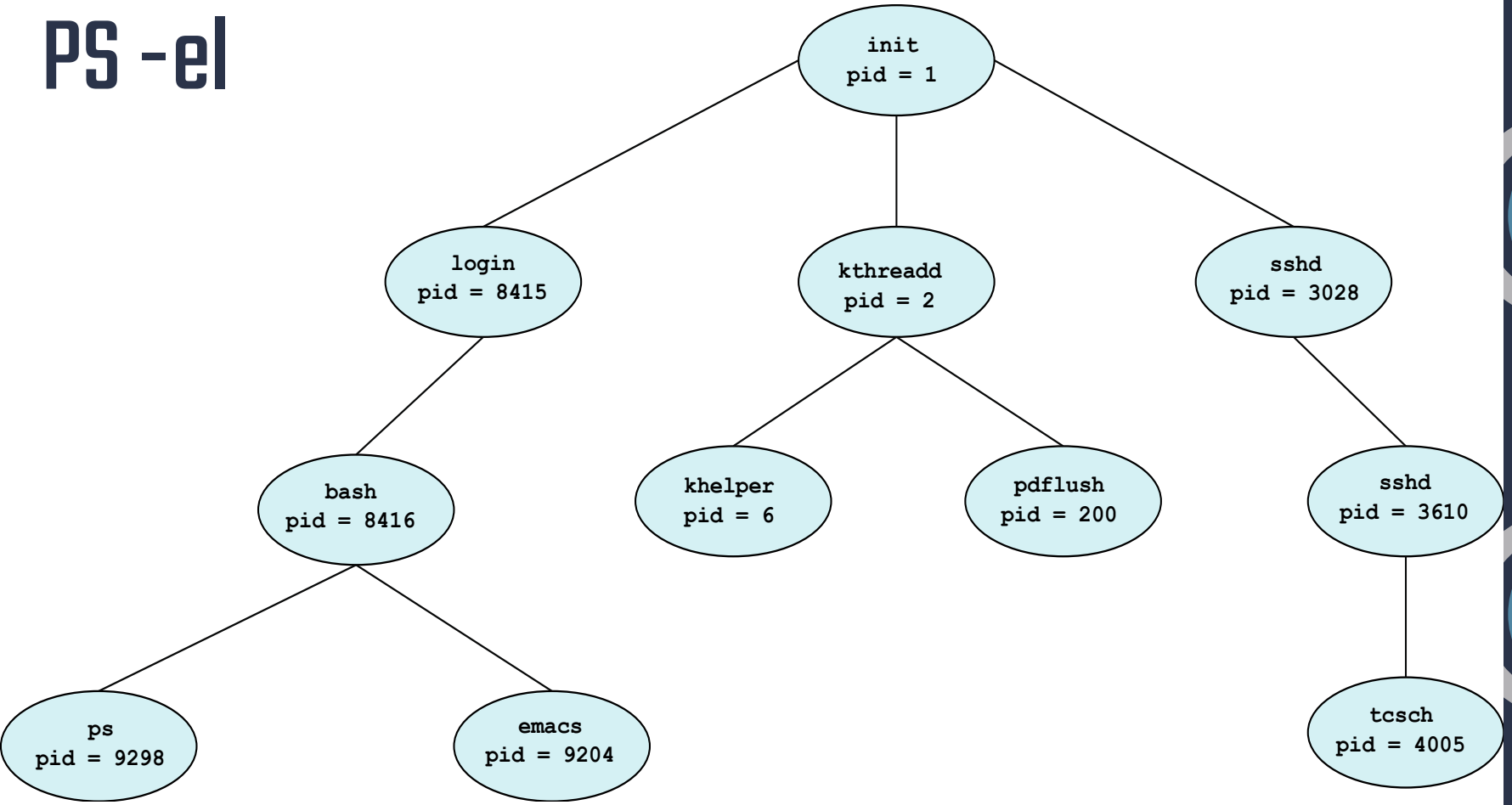
Cuando un proceso engendra a otro puede compartirle ciertos recursos.

Que recursos?

Que tipo de estructura de datos obtenemos de una relación padre hijo?



PS -el


init
pid = 1
login
pid = 8415
kthreadd
pid = 2
sshd
pid = 3028
bash
pid = 8416
khelper
pid = 6
pdflush
pid = 200
sshd
pid = 3610
ps
pid = 9298
emacs
pid = 9204
tcsch
pid = 4005

LXTerminal

File Edit Tabs Help

```
top - 05:54:35 up  2:43,  3 users,  load average: 0.00, 0.01, 0.05
Tasks:  96 total,   1 running,  95 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.7 us,  0.3 sy,  0.0 ni, 99.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem:    509428 total,   483252 used,    26176 free,    28132 buffers
KiB Swap:  2333960 total,        0 used,  2333960 free,   373176 cached

  PID USER      PR  NI  VIRT  RES  SHR S  %CPU %MEM    TIME+  COMMAND
 2175 root      20   0 67220  18m 7668 S   0.3  3.7   2:15.03 Xorg
 2332 knoppix   20   0 66516 9492 7692 S   0.3  1.9   0:00.77 metacity
 2350 knoppix   20   0 53356  13m 8904 S   0.3  2.7   0:06.30 lxpanel
 2445 knoppix   20   0 56420 9296 7320 S   0.3  1.8   0:12.01 x-terminal-emul
    1 root      20   0  2364  660  564 S   0.0  0.1   0:00.87 init
    2 root      20   0     0    0    0 S   0.0  0.0   0:00.00 kthreadd
    3 root      20   0     0    0    0 S   0.0  0.0   0:00.37 ksoftirqd/0
    5 root       0 -20     0    0    0 S   0.0  0.0   0:00.00 kworker/0:0H
    6 root      20   0     0    0    0 S   0.0  0.0   0:00.00 kworker/u:0
    7 root       0 -20     0    0    0 S   0.0  0.0   0:00.00 kworker/u:0H
    8 root      rt   0     0    0    0 S   0.0  0.0   0:00.00 migration/0
    9 root      20   0     0    0    0 S   0.0  0.0   0:00.62 rcu_preempt
   10 root      20   0     0    0    0 S   0.0  0.0   0:00.00 rcu_bh
   11 root      20   0     0    0    0 S   0.0  0.0   0:00.00 rcu_sched
   12 root       0 -20     0    0    0 S   0.0  0.0   0:00.00 khelper
   13 root       0 -20     0    0    0 S   0.0  0.0   0:00.00 netns
  433 root      20   0     0    0    0 S   0.0  0.0   0:00.00 bdi-default
```

INIT VS SYSTEMD

oscreader@OSC: ~/Desktop/Labs/4

File Edit View Search Terminal Help

```
top - 12:03:12 up 36 min,  2 users,  load average: 0.13, 0.09, 0.18
Tasks: 133 total,   1 running, 132 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2.3 us,  1.0 sy,  0.0 ni, 96.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem:    771744 total,   708376 used,    63368 free,    94396 buffers
KiB Swap:   392188 total,    96704 used,   295484 free.   262632 cached Mem

  PID USER      PR  NI    VIRT    RES    SHR S %CPU %MEM     TIME+ COMMAND
 1061 oscread+  20   0  427528 127516  45388 S  2.7 16.5   2:34.22 gnome-shell
  507 root      20   0  111432  25712   6052 S  0.7  3.3   0:39.95 Xorg
 1536 oscread+  20   0   65408  24196  19748 S  0.3  3.1   0:02.04 gnome-termi+
 1716 oscread+  20   0    5080   2552   2188 R  0.3  0.3   0:00.11 top
    1 root      20   0   23956   2660   1724 S  0.0  0.3   0:01.10 systemd
    2 root      20   0       0      0      0 S  0.0  0.0   0:00.00 kthreadd
    3 root      20   0       0      0      0 S  0.0  0.0   0:00.36 ksoftirqd/0
    5 root       0 -20       0      0      0 S  0.0  0.0   0:00.00 kworker/0:0H
    6 root      20   0       0      0      0 S  0.0  0.0   0:00.23 kworker/u2:0
    7 root      20   0       0      0      0 S  0.0  0.0   0:00.67 rcu_sched
    8 root      20   0       0      0      0 S  0.0  0.0   0:00.00 rcu_bh
    9 root      rt   0       0      0      0 S  0.0  0.0   0:00.00 migration/0
   10 root      rt   0       0      0      0 S  0.0  0.0   0:00.02 watchdog/0
   11 root       0 -20       0      0      0 S  0.0  0.0   0:00.00 khelper
   12 root      20   0       0      0      0 S  0.0  0.0   0:00.00 kdevtmpfs
   13 root       0 -20       0      0      0 S  0.0  0.0   0:00.00 netns
   14 root      20   0       0      0      0 S  0.0  0.0   0:00.00 khungtaskd
```

INIT VS SYSTEMD

Reglas para traer hijos al mundo

- Cuando un proceso crea a otro, el proceso padre puede continuar ejecutándose concurrentemente o esperar a su(s) hijo(s).
    - O ambos. Es posible que el padre ejecute ciertas acciones y luego se detenga a esperar (a uno o más de sus hijos).

- Ejecutar un programa requerirá, además de crear un proceso nuevo, cargar el código y la data del programa en cuestión.
    - En sistemas UNIX se usa la llamada exec() que reemplaza al programa actual.
        - La ejecución de programas conlleva crear un proceso (con fork()) y cargarle encima otro programa (con exec()).

Reglas para traer hijos al mundo

- Cuando un proceso crea a otro, el proceso hijo deberá ser asignado con un PID unico

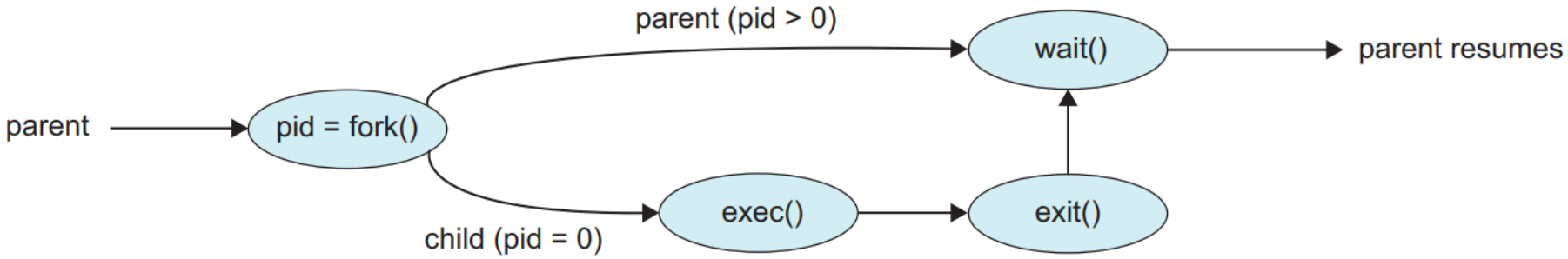
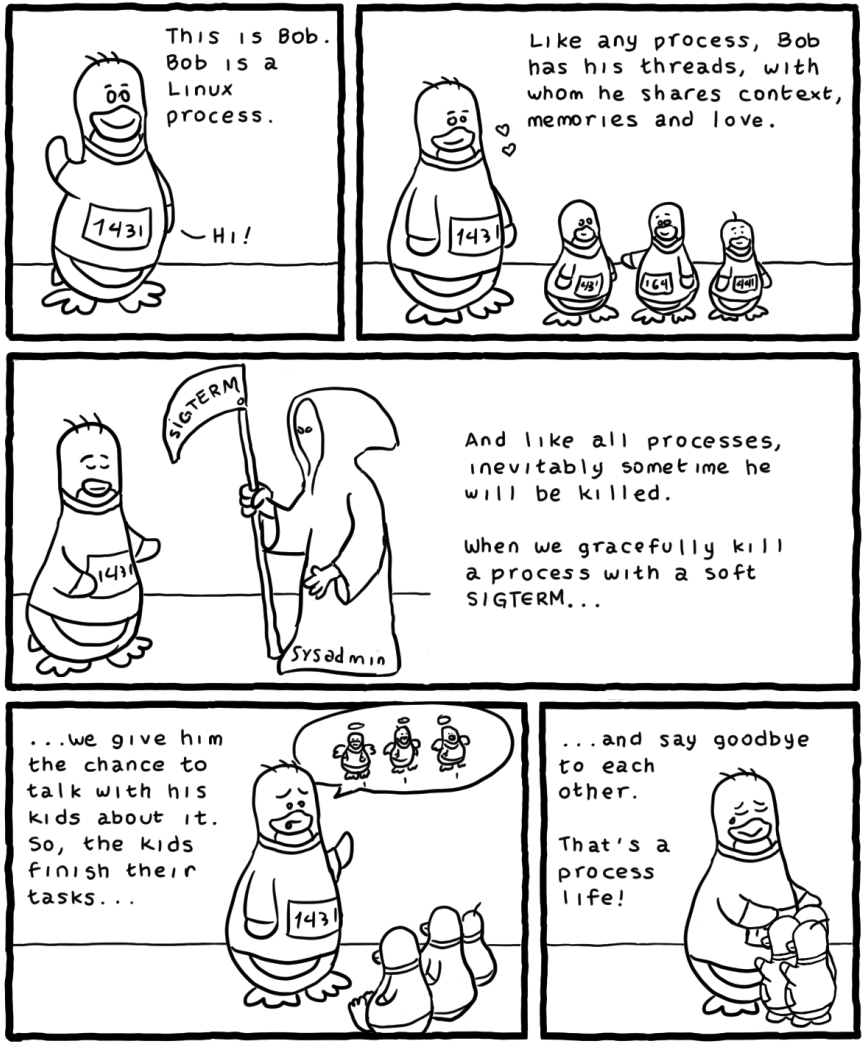


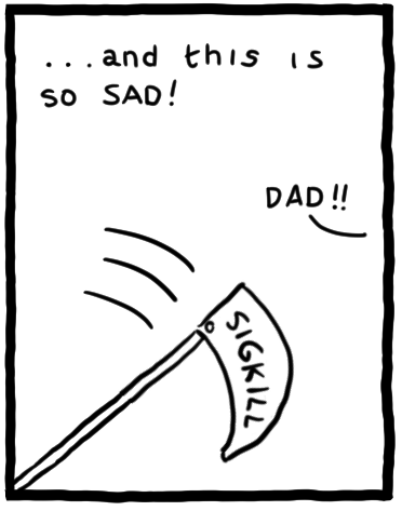
Figure 3.10 Process creation using the `fork()` system call.

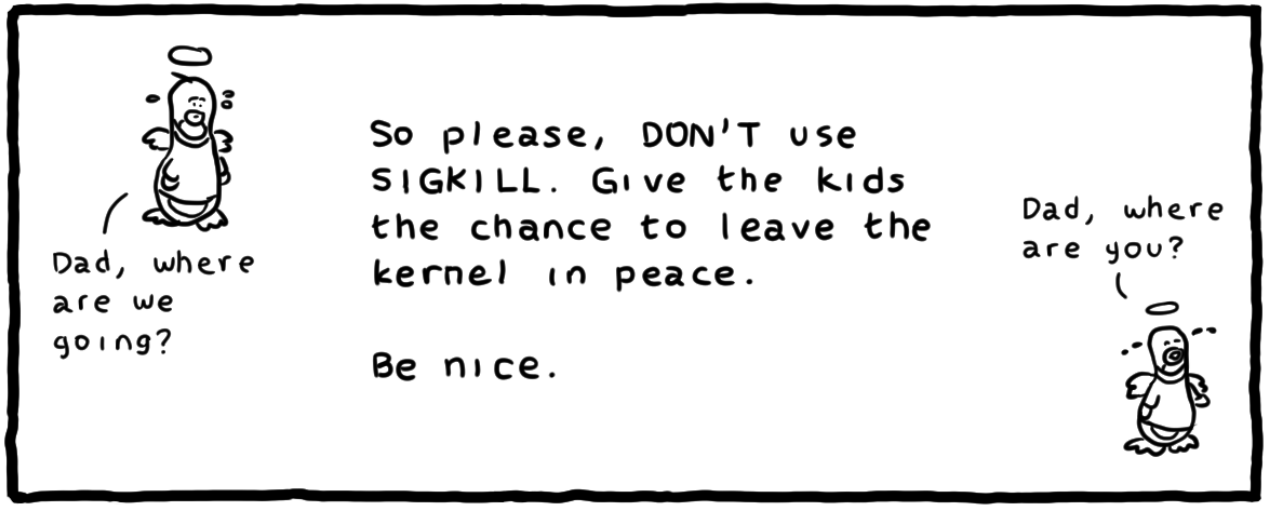
Terminacion de procesos

- Un proceso puede terminar su ejecución voluntaria o involuntariamente.
  - El OS provee una syscall (en sistemas UNIX, exit()) para terminación voluntaria.
  - La terminación invountaria se da caso de error fatal (e.g., operaciones no permitidas) o mediante otro proceso (que requerirá ciertos privilegios).
- Un proceso padre puede matar a sus hijos (abort()) o esperar a que terminen (wait()).
  - Es posible que un proceso hijo termine antes de que el padre llegue a esperarlo. En este caso, el hijo queda zombie.
- Las posibilidades cuando un proceso padre termina son:
  - Abandonar a los hijos (quedan huérfanos).
- Los huérfanos suelen ser adoptados por init.
  - Llevarse a los hijos a la tumba (terminación en cascada).

ACOMPAÑAME A VER
ESTA TRISTE HISTORIA
memegenerator.net


This is Bob. Bob is a Linux process.
1431
Hi!
Like any process, Bob has his threads, with whom he shares context, memories and love.
1431
481
1641
4441
SIGTERM
1431
Sysadmin
And like all processes, inevitably sometime he will be killed.
When we gracefully kill a process with a soft SIGTERM...
...we give him the chance to talk with his kids about it. So, the kids finish their tasks...
1431
...and say goodbye to each other.
That's a process life!





Daniel Stori {turnoff.us}

ASISTENTE DE DIRECCION
Enrique Becerra
ASISTENTES DE PRODUCCION
Dagoberto Vidal

03 COMUNICACION ENTRE PROCESOS

ING. JUAN LUIS GARCIA

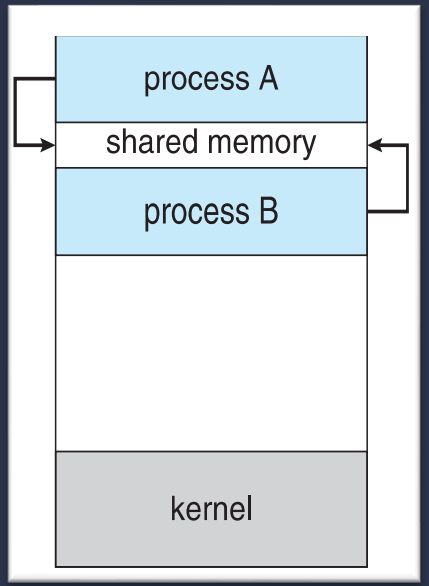
Nos conviene tener comunicación entre procesos?

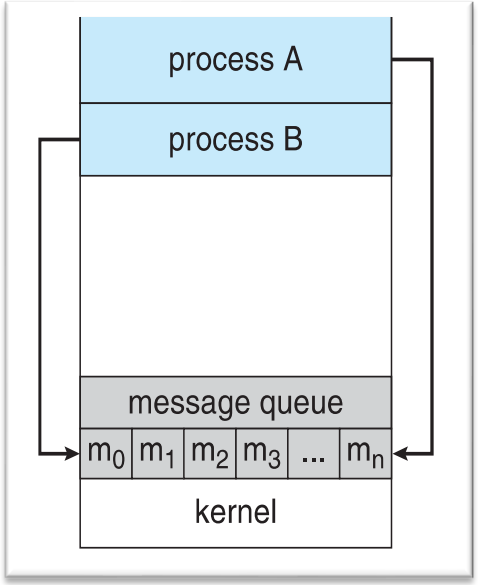
Motivación para comunicación entre procesos:

- Para compartir información y recursos, hay que coordinarse.
- Para modularizar un sistema y reutilizar código, procesos pueden requerir servicios entre sí.
- Para “encadenar” tareas y usar el output de un proceso como input de otro.
- Para sincronizar ejecución de procesos y establecer un orden entre tareas.

Shared Memory

Procesos tienen acceso a un mismo espacio de memoria en el que pueden leer y escribir.





Queue

Procesos envían información a otros siguiendo un protocolo.

Shared Memory

- ## El concepto atras de esto...

  Un proceso solicita un espacio de memoria compartida que es adherido al espacio de memoria de los procesos en comunicación.

- ## Rapidez

  Suele ser más rápido que paso de mensajes porque requiere menos intervención del sistema operativo.

- ## Suele ser más problemático que paso de mensajes

  Es más difícil de implementar, especialmente en sistemas distribuidos.

  Nos enfrenta a conflictos de acceso y sincronización.

Queue

- ## El concepto atras de esto...

  El paso de mensajes prescinde de una región de memoria compartida en favor de un enlace de comunicación (communication link).

- ## Operaciones basicas

  El modelo de paso de mensajes emplea al menos dos operaciones básicas send() receive()

- ## Nos tenemos que poner de acuerdo

  La implementación del link y las operaciones de envío y recepción determina la forma en la que la comunicación se lleva a cabo.
  Comunicación directa o indirecta, Comunicación síncrona o asíncrona ,Buffering implícito o explícito.

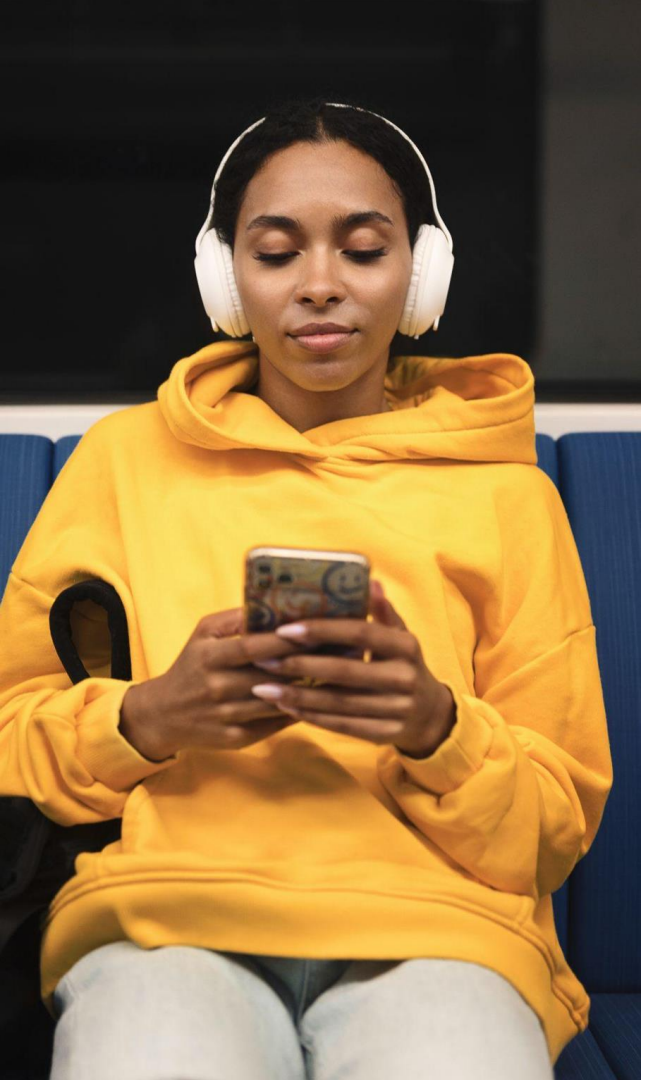
Comunicación directa e indirecta

Directa	Indirecta
• Simétrica implica que los procesos especifican a quién envían y de quién reciben. • Asimétrica requiere únicamente que el emisor especifique el destino de su mensaje. • Los procesos sólo necesitan conocer la identidad del otro. Se establece exactamente un link entre exactamente dos procesos.	• El link se establece entre pares de procesos que comparten el buzón. • Un buzón puede facilitar la comunicación entre más de dos procesos. • Dos procesos se pueden comunicar por medio de más de un buzón. • ¿Qué pasa si tres procesos comparten un buzón y dos procesos intentan recibir a la vez un mismo mensaje dejado por el tercer proceso?

Sincronizacion

- La sincronía, en este contexto, se refiere al uso de bloqueos durante la comunicación.
- No necesariamente se empean las tecnologías en pareja. Se pueden hacer combinaciones.
- Tener tanto send como receive bloqueadoras presenta un rendezvous entre emisor y receptor.

Síncrona	Asíncrona
• Envío/recepción bloqueadora: el proceso emisor/receptor se bloquea hasta que el mensaje enviado/esperado sea recibido	• Envío/recepción no-bloqueadora: el emisor continúa trabajando luego de enviar, y el receptor recibe null si no hay nada que recibir.



BUFFERS

Los casos son tres:

- Sin buffering: cola de capacidad cero. Obliga al emisor a bloquearse hasta que lo enviado sea recibido.
- Buffering automático (finito): la cola tiene capacidad limitada. El emisor se bloquea al enviar un mensaje cuando la cola destino está llena. Se desbloquea cuando se libera espacio en la cola.
- Buffering automático (ilimitado): el emisor nunca se bloquea porque siempre hay espacio en la cola.

PRODUCER VS CONSUMER

De que estamos hablando?

- El problema Productor/Consumidor consiste en el acceso concurrente por parte de procesos productores y procesos consumidores sobre un recurso común que resulta ser un buffer de elementos. Los productores tratan de introducir elementos en el buffer de uno en uno, y los consumidores tratan de extraer elementos de uno en uno.

Situación

- Imagina una fábrica en la que hay trabajadores (productores) que fabrican productos y trabajadores (consumidores) que compran y consumen esos productos.

Problema

- Los productores deben colocar los productos en una mesa (buffer) para que los consumidores los recojan y los utilicen.
- Si la mesa está llena y no hay espacio para más productos, los productores deben esperar hasta que haya espacio disponible.
- Si la mesa está vacía y no hay productos, los consumidores deben esperar hasta que haya productos disponibles.

Desafío

- La tarea es coordinar a los productores y consumidores para que no intenten colocar o tomar productos al mismo tiempo, evitando así conflictos y asegurando que la mesa no se llene o se vacíe de manera incorrecta.

Solución

Desafío

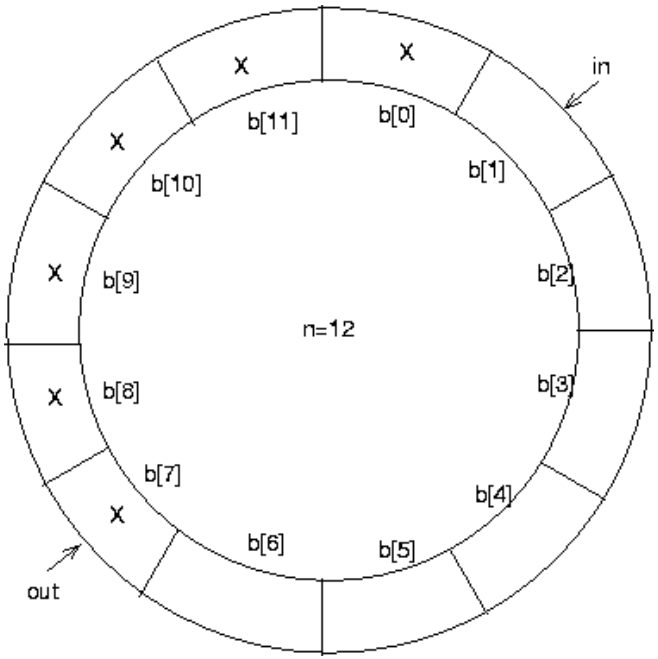
- La tarea es coordinar a los productores y consumidores para que no intenten colocar o tomar productos al mismo tiempo, evitando así conflictos y asegurando que la mesa no se llene o se vacíe de manera incorrecta.

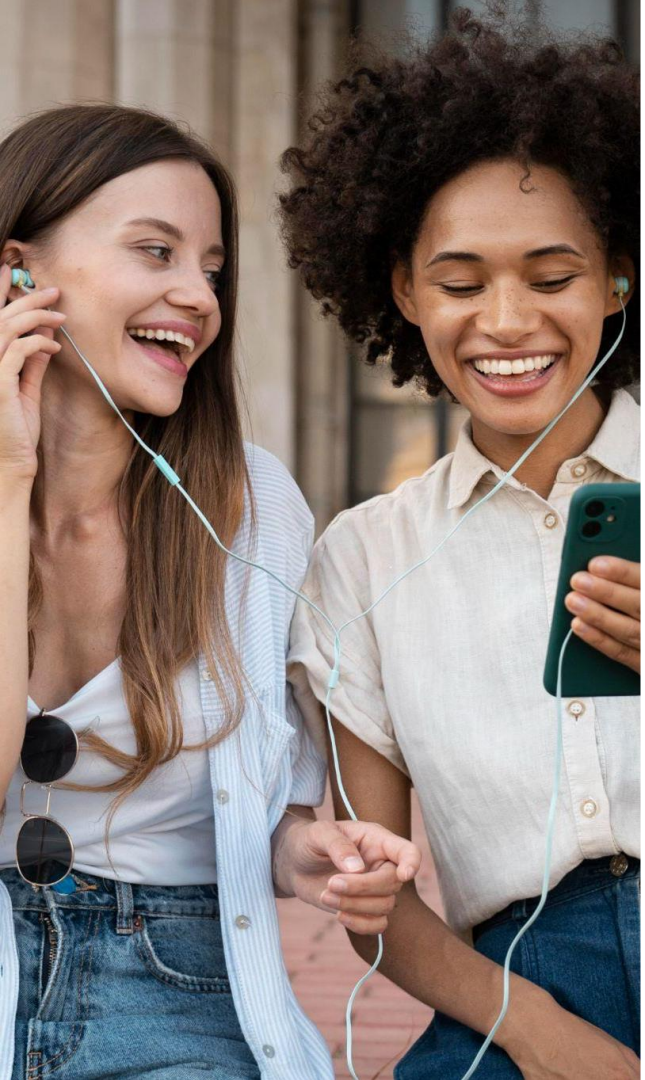
Solución

- Se necesitan mecanismos de sincronización, como semáforos o mutex, para garantizar que los productores y consumidores se comuniquen adecuadamente y eviten situaciones problemáticas como la sobreproducción o la falta de productos.

Resumen y como se implementa

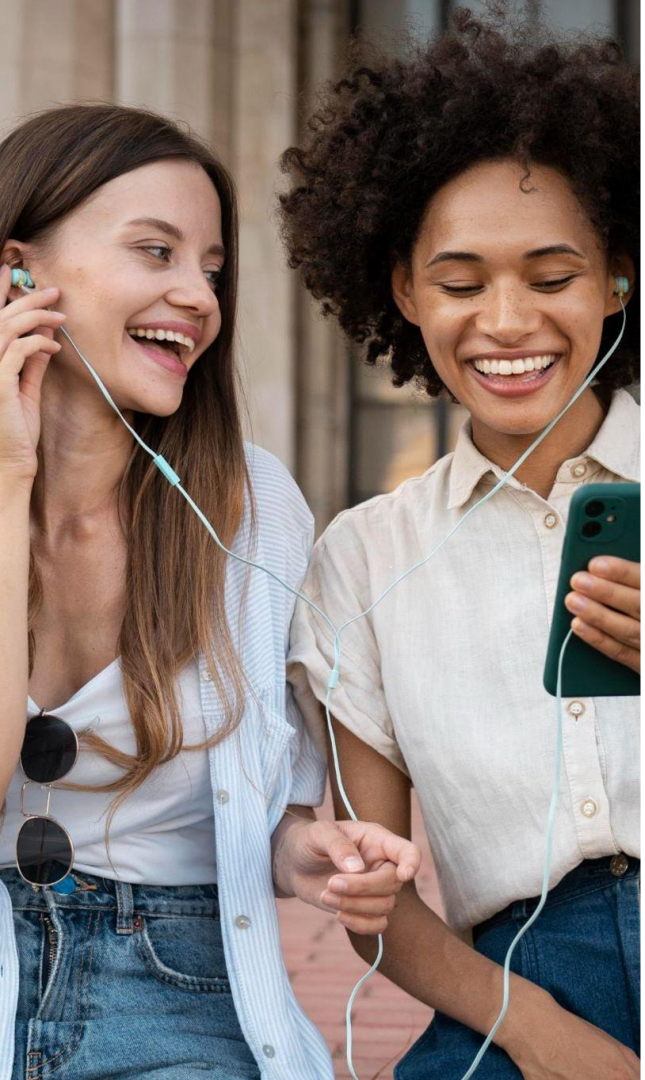
- En el problema del productor y el consumidor, el reto es gestionar la interacción entre dos tipos de trabajadores (productores y consumidores) para evitar problemas como la sobreproducción o la falta de productos. La coordinación adecuada es esencial para garantizar un funcionamiento suave y eficiente del sistema. Este problema es relevante en sistemas operativos y programación concurrente, donde múltiples procesos o hilos interactúan entre sí.



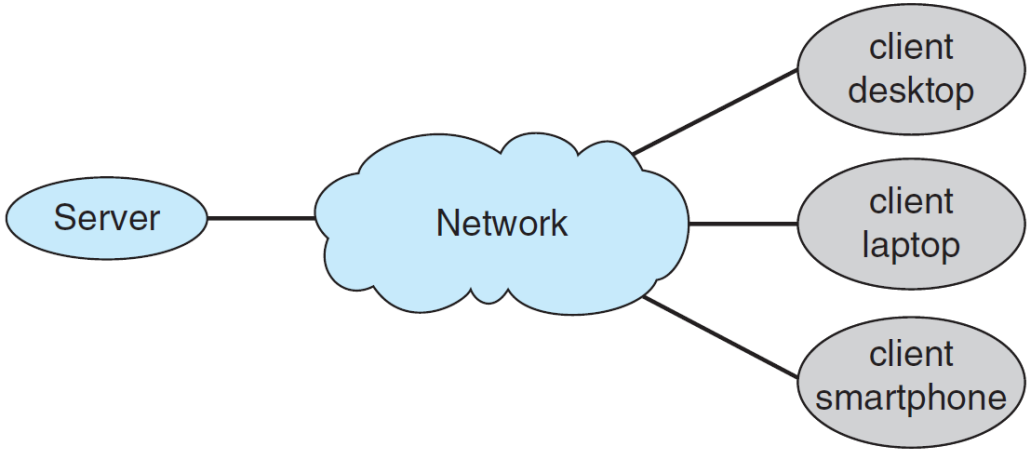


Sistemas cliente servidor

Enfoquemonos en la forma de COMUNICACION



Sistemas cliente servidor



SOCKETS

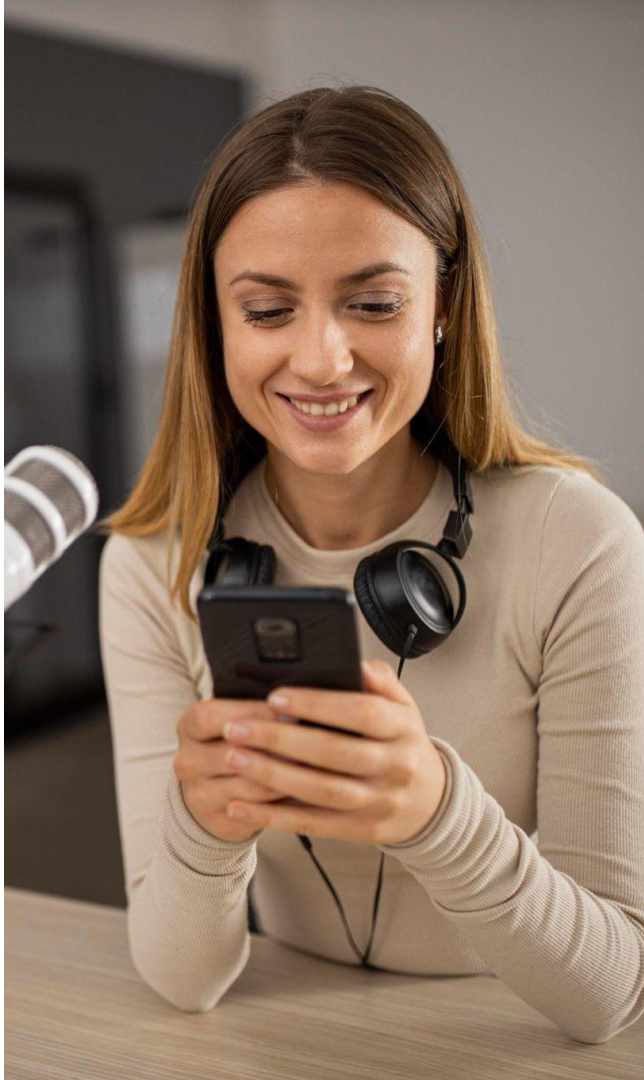
Un socket es la tia chismosa de la familia...

- Un socket es un punto (endpoint) de comunicación para un sistema.
- Frecuentemente usados para comunicación entre computadoras a través de una red (mediante el protocolo TCP).
- Se identifican con una dirección IP y un puerto de software (e.g., 161.25.19.8:1625).
- La dirección 127.0.0.1 es conocida como el loopback: una autoreferencia.
- Los puertos bien conocidos (well known ports) tienen números del 1 al 1024. Suelen ser usados para tipos específicos de comunicación. Los sockets son un recurso de conexión provisto por el sistema operativo.
- Un proceso solicita la creación de un socket para escuchar cierto puerto en el sistema.
- El sistema operativo crea el socket y provee una referencia al proceso solicitante.

Un socket es la tia chismosa de la familia...

Thanks!

Do you have any questions?

jlgarcia@uvg.edu.gt
+502 49740808
Uvg.edu.gt

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**